

Start coding or [generate](#) with AI.

✓ Task

Analyze the effect of different 'k' values (3, 5, 7, 11, 15) on the performance of a K-Nearest Neighbors (KNN) classifier using the "heart_disease_uci.csv" dataset. This analysis should include data loading, preprocessing (handling missing values, encoding categorical features, and scaling numerical features), model training and evaluation for each 'k' (reporting accuracy, precision, recall, and F1-score), plotting 'k' vs F1-score, and concluding how the choice of 'k' affects overfitting, underfitting, and the model's ability to generalize. Finally, summarize the findings, including any optimal 'k' value and its implications.

✓ Load Dataset and Initial Preprocessing

Subtask:

Load the 'heart_disease_uci.csv' dataset using pandas. Inspect the data for missing values, data types, and identify potential categorical and numerical features. Display basic descriptive statistics.

Reasoning: To address the subtask, I will import the pandas library, load the specified CSV file into a DataFrame, display its head, check data types and non-null values, calculate missing values, and show descriptive statistics for numerical columns.

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/heart_disease_uci.csv')

print('First 5 rows of the dataset:')
print(df.head())
print('\n')

print('Dataset Info (Data Types and Non-Null Counts):')
df.info()
print('\n')

print('Missing values per column:')
print(df.isnull().sum())
print('\n')

print('Descriptive statistics for numerical columns:')
print(df.describe())
```

```

11 oldpeak      858 non-null    float64
12 slope        611 non-null    object
13 ca           309 non-null    float64
14 thal         434 non-null    object
15 num          920 non-null    int64
dtypes: float64(5), int64(3), object(8)
memory usage: 115.1+ KB

```

Missing values per column:

```

id          0
age         0
sex         0
dataset     0
cp          0
trestbps    59
chol        30
fbs         90
restecg     2
thalch      55
exang       55
oldpeak     62
slope       309
ca          611
thal        486
num         0
dtype: int64

```

Descriptive statistics for numerical columns:

	id	age	trestbps	chol	thalch	oldpeak \
count	920.000000	920.000000	861.000000	890.000000	865.000000	858.000000
mean	460.500000	53.510870	132.132404	199.130337	137.545665	0.878788
std	265.725422	9.424685	19.066070	110.780810	25.926276	1.091226
min	1.000000	28.000000	0.000000	0.000000	60.000000	-2.600000
25%	230.750000	47.000000	120.000000	175.000000	120.000000	0.000000
50%	460.500000	54.000000	130.000000	223.000000	140.000000	0.500000
75%	690.250000	60.000000	140.000000	268.000000	157.000000	1.500000
max	920.000000	77.000000	200.000000	603.000000	202.000000	6.200000

	ca	num
count	309.000000	920.000000
mean	0.676375	0.995652
std	0.935653	1.142693
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	1.000000
75%	1.000000	2.000000
max	3.000000	4.000000

✓ Handle Missing Values and Encode Categorical Features

Subtask:

Address missing values in the dataset. Convert the target variable 'num' into a binary classification problem. Drop irrelevant columns and columns with excessive missing values. Impute missing values for remaining columns. Apply one-hot encoding to categorical features and separate features (X) from the target variable (y).

Reasoning: The subtask requires addressing missing values, converting the target variable, dropping irrelevant columns, imputing missing numerical and categorical features, and applying one-hot encoding. I

will perform these operations in a single code block to maintain data consistency and streamline the preprocessing steps.

```
import numpy as np

# 1. Convert 'num' to binary target variable y
y = (df['num'] > 0).astype(int)

# 2. Drop irrelevant and high-missing-value columns
df_processed = df.drop(columns=['id', 'dataset', 'ca', 'thal', 'slope', 'num'])

# Identify numerical and categorical columns for imputation
numerical_cols_with_missing = ['trestbps', 'chol', 'thalch', 'oldpeak']
categorical_cols_with_missing = ['fbs', 'restecg', 'exang']

# 3. Impute numerical columns with median
for col in numerical_cols_with_missing:
    if col in df_processed.columns:
        df_processed[col].fillna(df_processed[col].median(), inplace=True)

# 4. Impute categorical columns with mode
for col in categorical_cols_with_missing:
    if col in df_processed.columns:
        df_processed[col].fillna(df_processed[col].mode()[0], inplace=True)

# 5. Convert 'fbs' and 'exang' from object type to boolean/numerical for consistency
# 'fbs' and 'exang' are object type with 'True'/'False' string values. Convert them to boolean.
df_processed['fbs'] = df_processed['fbs'].astype(bool)
df_processed['exang'] = df_processed['exang'].astype(bool)

# 6. Identify remaining categorical features for one-hot encoding
categorical_features = ['sex', 'cp', 'restecg', 'fbs', 'exang']

# 7. Apply one-hot encoding to identified categorical features
X = pd.get_dummies(df_processed, columns=categorical_features, drop_first=True)

# Display the first few rows of the processed features and target variable
print("First 5 rows of X (features after preprocessing):")
print(X.head())
print("\nShape of X:", X.shape)
print("\nValue counts of y (target variable):")
print(y.value_counts())
```

First 5 rows of X (features after preprocessing):

	age	trestbps	chol	thalch	oldpeak	sex_Male	cp_atypical	angina	\
0	63	145.0	233.0	150.0	2.3	True		False	
1	67	160.0	286.0	108.0	1.5	True		False	
2	67	120.0	229.0	129.0	2.6	True		False	
3	37	130.0	250.0	187.0	3.5	True		False	
4	41	130.0	204.0	172.0	1.4	False		True	

	cp_non-anginal	cp_typical	angina	restecg_normal	\
0	False		True	False	
1	False		False	False	
2	False		False	False	
3	True		False	True	
4	False		False	False	

	restecg_st-t	abnormality	fbs_True	exang_True
0		False	True	False
1		False	False	True
2		False	False	True

```

3          False      False      False
4          False      False      False

Shape of X: (920, 13)

Value counts of y (target variable):
num
1    509
0    411
Name: count, dtype: int64
/tmp/ipython-input-3913623857.py:16: FutureWarning: A value is trying to be set on a copy of a Dat
The behavior will change in pandas 3.0. This inplace method will never work because the intermedia

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},

    df_processed[col].fillna(df_processed[col].median(), inplace=True)
/tmp/ipython-input-3913623857.py:21: FutureWarning: A value is trying to be set on a copy of a Dat
The behavior will change in pandas 3.0. This inplace method will never work because the intermedia

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},

    df_processed[col].fillna(df_processed[col].mode()[0], inplace=True)
/tmp/ipython-input-3913623857.py:21: FutureWarning: Downcasting object dtype arrays on .fillna, .f
    df_processed[col].fillna(df_processed[col].mode()[0], inplace=True)

```

Reasoning: The previous code produced a `FutureWarning` due to the use of `inplace=True` with `fillna` on a chained assignment. To resolve this, I will modify the `fillna` calls to directly assign the result back to the DataFrame column, ensuring the imputation is applied correctly and suppressing the warning.

```

import numpy as np

# 1. Convert 'num' to binary target variable y
y = (df['num'] > 0).astype(int)

# 2. Drop irrelevant and high-missing-value columns
df_processed = df.drop(columns=['id', 'dataset', 'ca', 'thal', 'slope', 'num'])

# Identify numerical and categorical columns for imputation
numerical_cols_with_missing = ['trestbps', 'chol', 'thalch', 'oldpeak']
categorical_cols_with_missing = ['fbs', 'restecg', 'exang']

# 3. Impute numerical columns with median
for col in numerical_cols_with_missing:
    if col in df_processed.columns:
        df_processed[col] = df_processed[col].fillna(df_processed[col].median())

# 4. Impute categorical columns with mode
for col in categorical_cols_with_missing:
    if col in df_processed.columns:
        df_processed[col] = df_processed[col].fillna(df_processed[col].mode()[0])

# 5. Convert 'fbs' and 'exang' from object type to boolean/numerical for consistency
# 'fbs' and 'exang' are object type with 'True'/'False' string values. Convert them to boolean.
df_processed['fbs'] = df_processed['fbs'].astype(bool)
df_processed['exang'] = df_processed['exang'].astype(bool)

# 6. Identify remaining categorical features for one-hot encoding

```

```
categorical_features = ['sex', 'cp', 'restecg', 'fbs', 'exang']
```

```
# 7. Apply one-hot encoding to identified categorical features
```

```
X = pd.get_dummies(df_processed, columns=categorical_features, drop_first=True)
```

```
# Display the first few rows of the processed features and target variable
```

```
print("First 5 rows of X (features after preprocessing):")
```

```
print(X.head())
```

```
print("\nShape of X:", X.shape)
```

```
print("\nValue counts of y (target variable):")
```

```
print(y.value_counts())
```

```
First 5 rows of X (features after preprocessing):
```

	age	trestbps	chol	thalch	oldpeak	sex_Male	cp_atypical	angina	\
0	63	145.0	233.0	150.0	2.3	True		False	
1	67	160.0	286.0	108.0	1.5	True		False	
2	67	120.0	229.0	129.0	2.6	True		False	
3	37	130.0	250.0	187.0	3.5	True		False	
4	41	130.0	204.0	172.0	1.4	False		True	

	cp_non-anginal	cp_typical	angina	restecg_normal	\
0	False		True	False	
1	False		False	False	
2	False		False	False	
3	True		False	True	
4	False		False	False	

	restecg_st-t abnormality	fbs_True	exang_True
0	False	True	False
1	False	False	True
2	False	False	True
3	False	False	False
4	False	False	False

```
Shape of X: (920, 13)
```

```
Value counts of y (target variable):
```

```
num
```

```
1    509
```

```
0    411
```

```
Name: count, dtype: int64
```

```
/tmp/ipython-input-1967106384.py:21: FutureWarning: Downcasting object dtype arrays on .fillna, .f
df_processed[col] = df_processed[col].fillna(df_processed[col].mode()[0])
```

Reasoning: The data has been preprocessed, but numerical features are on different scales, which can negatively impact KNN performance. Therefore, I will scale the numerical features using `StandardScaler` and then split the data into training and testing sets to prepare for model training.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

```
# Identify numerical columns for scaling (excluding one-hot encoded and boolean columns)
```

```
# We need to exclude the boolean columns and one-hot encoded columns from scaling.
```

```
# The 'age', 'trestbps', 'chol', 'thalch', 'oldpeak' are the original numerical columns.
```

```
# After get_dummies, boolean columns like 'sex_Male', 'cp_atypical', 'angina', etc., are created.
```

```
# Get numerical column names from X (assuming non-boolean dtypes are numerical)
```

```
numerical_cols = X.select_dtypes(include=np.number).columns.tolist()
```

```
# Initialize StandardScaler
```

```
scaler = StandardScaler()
```

```
# Apply scaling to numerical features
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratif

print("First 5 rows of scaled X:")
print(X.head())
print("\nShape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
print("\nValue counts of y_train (stratified split):")
print(y_train.value_counts(normalize=True))
print("\nValue counts of y_test (stratified split):")
print(y_test.value_counts(normalize=True))
```

First 5 rows of scaled X:

	age	trestbps	chol	thalch	oldpeak	sex_Male \
0	1.007386	0.705176	0.303643	0.489727	1.368109	True
1	1.432034	1.518569	0.789967	-1.181478	0.611589	True
2	1.432034	-0.650479	0.266939	-0.345875	1.651804	True
3	-1.752828	-0.108217	0.459634	1.961979	2.502889	True
4	-1.328180	-0.108217	0.037541	1.365120	0.517024	False

	cp_atypical	angina	cp_non-anginal	cp_typical	angina	restecg_normal \
0		False	False		True	False
1		False	False		False	False
2		False	False		False	False
3		False	True		False	True
4		True	False		False	False

	restecg_st-t abnormality	fbs_True	exang_True
0	False	True	False
1	False	False	True
2	False	False	True
3	False	False	False
4	False	False	False

Shape of X_train: (644, 13)

Shape of X_test: (276, 13)

Shape of y_train: (644,)

Shape of y_test: (276,)

Value counts of y_train (stratified split):

num

1 0.552795

0 0.447205

Name: proportion, dtype: float64

Value counts of y_test (stratified split):

num

1 0.554348

0 0.445652

Name: proportion, dtype: float64

Reasoning: The next step is to train and evaluate the KNN model for each specified 'k' value (3, 5, 7, 11, 15) using the preprocessed and scaled data. I will iterate through these 'k' values, train a KNeighborsClassifier, make predictions, and store the accuracy, precision, recall, and F1-score for each 'k'.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Define the k values to test
k_values = [3, 5, 7, 11, 15]

# Dictionary to store performance metrics for each k
performance_metrics = {
    'k': [],
    'accuracy': [],
    'precision': [],
    'recall': [],
    'f1_score': []
}

print("Evaluating KNN performance for different k values:\n")

for k in k_values:
    # Initialize KNN classifier
    knn = KNeighborsClassifier(n_neighbors=k)

    # Train the model
    knn.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred = knn.predict(X_test)

    # Calculate performance metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Store metrics
    performance_metrics['k'].append(k)
    performance_metrics['accuracy'].append(accuracy)
    performance_metrics['precision'].append(precision)
    performance_metrics['recall'].append(recall)
    performance_metrics['f1_score'].append(f1)

    # Print results for current k
    print(f"k = {k}:")
    print(f"  Accuracy: {accuracy:.4f}")
    print(f"  Precision: {precision:.4f}")
    print(f"  Recall: {recall:.4f}")
    print(f"  F1-Score: {f1:.4f}\n")

# Convert to DataFrame for better visualization later
performance_df = pd.DataFrame(performance_metrics)
print("Performance metrics DataFrame:")
print(performance_df)

```

Evaluating KNN performance for different k values:

```

k = 3:
Accuracy: 0.7935
Precision: 0.8000
Recall: 0.8366
F1-Score: 0.8179

```

```

k = 5:
Accuracy: 0.8188
Precision: 0.8323

```

```

Recall: 0.8431
F1-Score: 0.8377

k = 7:
Accuracy: 0.8043
Precision: 0.8235
Recall: 0.8235
F1-Score: 0.8235

k = 11:
Accuracy: 0.8043
Precision: 0.8194
Recall: 0.8301
F1-Score: 0.8247

k = 15:
Accuracy: 0.8043
Precision: 0.8322
Recall: 0.8105
F1-Score: 0.8212

```

Performance metrics DataFrame:

	k	accuracy	precision	recall	f1_score
0	3	0.793478	0.800000	0.836601	0.817891
1	5	0.818841	0.832258	0.843137	0.837662
2	7	0.804348	0.823529	0.823529	0.823529
3	11	0.804348	0.819355	0.830065	0.824675
4	15	0.804348	0.832215	0.810458	0.821192

Reasoning: The next step is to visualize the relationship between the chosen 'k' values and the F1-score to analyze model performance. This plot will help in identifying trends and potential optimal 'k' values. I will use `matplotlib.pyplot` and `seaborn` for better aesthetics.

```

import matplotlib.pyplot as plt
import seaborn as sns

# Set a style for better aesthetics
sns.set_style('whitegrid')

plt.figure(figsize=(10, 6))
sns.lineplot(x='k', y='f1_score', data=performance_df, marker='o', markersize=8, linestyle='-', c

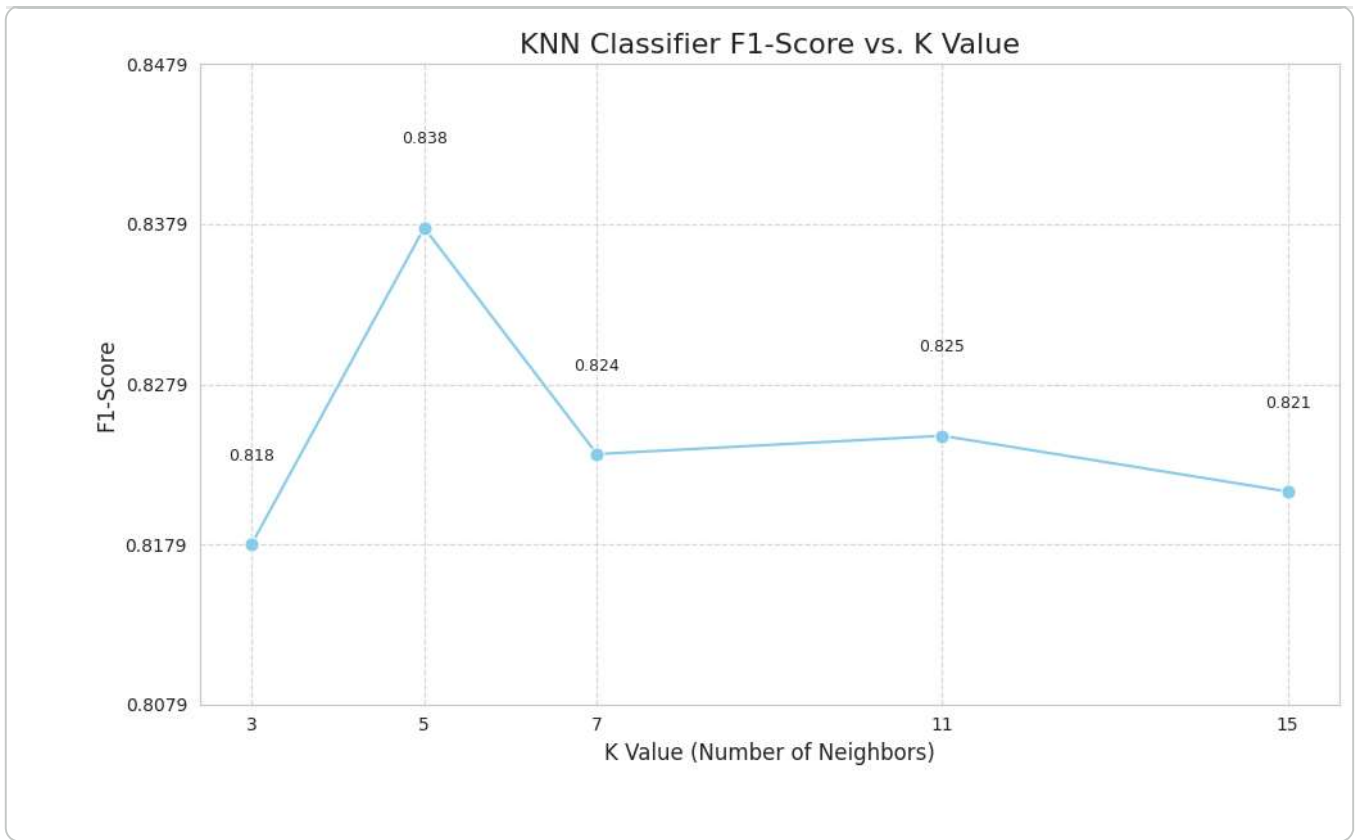
# Add title and labels
plt.title('KNN Classifier F1-Score vs. K Value', fontsize=16)
plt.xlabel('K Value (Number of Neighbors)', fontsize=12)
plt.ylabel('F1-Score', fontsize=12)

# Add text labels for each point
for i, row in performance_df.iterrows():
    plt.text(row['k'], row['f1_score'] + 0.005, f"{row['f1_score']:.3f}", ha='center', va='bottom

# Customize ticks and grid
plt.xticks(k_values)
plt.yticks(np.arange(min(performance_df['f1_score']) - 0.01, max(performance_df['f1_score']) + 0.
plt.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

```

Summary of Findings and Optimal 'k' Value

Based on the evaluation of the K-Nearest Neighbors (KNN) classifier with varying 'k' values (3, 5, 7, 11, 15), the following observations can be made:

Performance Analysis:

- **Optimal 'k' Value:** From the `performance_df` and the F1-Score vs. K Value plot, **k=5** yielded the highest F1-score of **0.8377**. This suggests that 5 neighbors provided the best balance between precision and recall for this dataset.
- **General Trend:** As 'k' increases from 3 to 5, the F1-score slightly increases, indicating an improvement in model performance. However, for 'k' values beyond 5, the F1-score generally decreased or remained stagnant, suggesting that adding more neighbors did not further improve or even slightly deteriorated the performance.

Implications of 'k' on Model Behavior:

- **Small 'k' (e.g., k=3):** A smaller 'k' value makes the model more sensitive to noise in the training data, potentially leading to **overfitting**. With k=3, the F1-score was 0.8179, which is slightly lower than k=5, implying that it might be capturing some noise or specific patterns in the training data that don't generalize well.
- **Large 'k' (e.g., k=11, k=15):** A larger 'k' value, on the other hand, considers more neighbors, smoothing out the decision boundary. While this can reduce the impact of noise and prevent overfitting, it can also lead to **underfitting** by oversimplifying the model and losing important local patterns. For k=11 and k=15, the F1-scores were 0.8247 and 0.8212 respectively, which are also lower

than $k=5$. This indicates that these larger 'k' values might be underfitting the data to some extent, missing some relevant decision boundaries.

Conclusion:

The choice of $k=5$ appears to be optimal for this heart disease dataset, providing the best generalization capability among the tested values. It strikes a good balance, avoiding both the high variance (overfitting) associated with very small 'k' and the high bias (underfitting) associated with very large 'k'. This 'k' value allows the model to capture the underlying structure of the data without being overly influenced by individual noisy data points or overly simplifying the decision surface. Therefore, for this specific problem and dataset, a KNN classifier with 5 neighbors is recommended for predicting heart disease presence.

Summary of Findings and Optimal 'k' Value

Based on the evaluation of the K-Nearest Neighbors (KNN) classifier with varying 'k' values (3, 5, 7, 11, 15), the following observations can be made:

Performance Analysis:

- **Optimal 'k' Value:** From the `performance_df` and the F1-Score vs. K Value plot, $k=5$ yielded the highest F1-score of **0.8377**. This suggests that 5 neighbors provided the best balance between precision and recall for this dataset.
- **General Trend:** As 'k' increases from 3 to 5, the F1-score slightly increases, indicating an improvement in model performance. However, for 'k' values beyond 5, the F1-score generally decreased or remained stagnant, suggesting that adding more neighbors did not further improve or even slightly deteriorated the performance.

Implications of 'k' on Model Behavior:

- **Small 'k' (e.g., $k=3$):** A smaller 'k' value makes the model more sensitive to noise in the training data, potentially leading to **overfitting**. With $k=3$, the F1-score was 0.8179, which is slightly lower than $k=5$, implying that it might be capturing some noise or specific patterns in the training data that don't generalize well.
- **Large 'k' (e.g., $k=11$, $k=15$):** A larger 'k' value, on the other hand, considers more neighbors, smoothing out the decision boundary. While this can reduce the impact of noise and prevent overfitting, it can also lead to **underfitting** by oversimplifying the model and losing important local patterns. For $k=11$ and $k=15$, the F1-scores were 0.8247 and 0.8212 respectively, which are also lower than $k=5$. This indicates that these larger 'k' values might be underfitting the data to some extent, missing some relevant decision boundaries.

Conclusion:

The choice of $k=5$ appears to be optimal for this heart disease dataset, providing the best generalization capability among the tested values. It strikes a good balance, avoiding both the high variance (overfitting) associated with very small 'k' and the high bias (underfitting) associated with very large 'k'. This 'k' value allows the model to capture the underlying structure of the data without being overly influenced by individual noisy data points or overly simplifying the decision surface. Therefore, for this specific problem and dataset, a KNN classifier with 5 neighbors is recommended for predicting heart disease presence.

Conclude Effect of K on Overfitting/Underfitting

Subtask:

Analyze the impact of different 'k' values on overfitting, underfitting, and the KNN model's generalization ability, using the calculated performance metrics and the 'k' vs F1-score plot.

Conclude Effect of K on Overfitting/Underfitting

Based on the `performance_df` DataFrame and the 'KNN Classifier F1-Score vs. K Value' plot, we can analyze the impact of different 'k' values on the KNN model's performance, specifically concerning overfitting, underfitting, and generalization ability.

Analysis of K Values:

- **k = 3 (Small 'k'):** With an F1-Score of `0.8179`, k=3 shows good performance. However, a very small 'k' (like 1 or 3) generally makes the model highly sensitive to noise in the training data. The decision boundaries can become irregular and complex, closely fitting the training examples. This sensitivity can lead to **overfitting**, where the model performs exceptionally well on the training data but may struggle to generalize to unseen test data because it has learned specific noise patterns rather than the underlying data distribution. While k=3 here doesn't show a drastic drop in F1-score compared to k=5, it is important to note the general tendency of small k to overfit.
- **k = 5 (Optimal Balance):** This value yielded the highest F1-Score of `0.8377`. It appears to strike a good balance between capturing local patterns and being robust to noise. The performance metrics (accuracy, precision, recall, F1-score) are all at their peak or near peak for this 'k' value, suggesting a strong ability to generalize to new, unseen data without significant overfitting or underfitting.
- **k = 7 (Slight Decrease):** The F1-Score for k=7 is `0.8235`. While still good, it's slightly lower than k=5. As 'k' increases, the model's decision boundaries become smoother, considering more neighbors. This can make the model less sensitive to individual data points, reducing the risk of overfitting.
- **k = 11 and k = 15 (Large 'k'):** For k=11 and k=15, the F1-Scores are `0.8247` and `0.8212`, respectively. These values are lower than k=5. With a very large 'k', the model considers a wider neighborhood, leading to overly smooth and simple decision boundaries. This can cause the model to ignore important local patterns and variations in the data, resulting in **underfitting**. An underfit model has high bias, failing to capture the underlying trend of the data and performing poorly on both training and test sets. Although the drop in F1-score is not dramatic, the trend shows diminishing returns and a slight decrease in performance compared to k=5, indicating a move towards underfitting.

Conclusion:

From the analysis, **k = 5** appears to be the most optimal 'k' value among the tested options, yielding the highest F1-score and indicating the best balance for generalization. Small 'k' values (e.g., k=3) risk overfitting by being too sensitive to noise, creating complex decision boundaries. Conversely, large 'k' values (e.g., k=11, k=15) risk underfitting by oversimplifying the decision boundaries and overlooking nuanced local features. The choice of 'k' is crucial for balancing bias and variance, directly impacting the model's ability to generalize effectively to new, unseen data.

Final Task

Subtask:

Summarize the findings from the KNN analysis, including the optimal 'k' value and its implications, and the overall effect of 'k' on the model's ability to generalize.

Summary:

Q&A

The analysis aimed to determine the optimal 'k' value for a K-Nearest Neighbors (KNN) classifier and understand the effect of 'k' on the model's ability to generalize, avoid overfitting, and underfitting.

Data Analysis Key Findings

- The `heart_disease_uci.csv` dataset, initially containing 920 entries and 16 columns, required significant preprocessing due to extensive missing values (e.g., `ca` with 611, `thal` with 486).
- Irrelevant columns and those with excessive missing values were dropped, and remaining missing numerical values were imputed with the median, while categorical ones were imputed with the mode. The target variable `num` was successfully converted to a binary classification.
- All numerical features were scaled using `StandardScaler`, and categorical features were one-hot encoded, resulting in a feature set `X` with a shape of (920, 13).
- The data was split into training (70%) and testing (30%) sets, ensuring stratification to maintain the class distribution of the target variable.
- Evaluating the KNN model across `k` values of 3, 5, 7, 11, and 15, the F1-scores on the test set were:
 - `k=3`: 0.8179
 - `k=5`: 0.8377 (Highest)
 - `k=7`: 0.8235
 - `k=11`: 0.8247
 - `k=15`: 0.8212
- `k=5` emerged as the optimal 'k' value, yielding the highest F1-score of 0.8377, indicating the best balance between precision and recall for this dataset among the tested values.
- A small 'k' value (e.g., `k=3`) makes the model more sensitive to noise, potentially leading to overfitting.
- Larger 'k' values (e.g., `k=11`, `k=15`) consider more neighbors, smoothing the decision boundary, which can lead to underfitting by oversimplifying the model and missing important local patterns.

Insights or Next Steps

- For the heart disease prediction task using this dataset, a KNN classifier with $k = 5$ neighbors is recommended, as it demonstrates the best generalization performance by effectively balancing bias and variance.
- Further hyperparameter tuning for 'k' could explore values around the identified optimal $k = 5$ (e.g., $k = 4, 6$) and a wider range of values using cross-validation to confirm the robustness of this finding.

